

락 경합에 의한 트랜잭션 대기 시간에 대한 연구

유혜진[○] 문성재[○] 박종혁^{*} 이상원⁺ 엄영익[○]

성균관대학교[○], 한국외국어대학교^{*}, 서울대학교⁺

{gpwls4306, dakoyaki}@skku.edu, jonghyeok.par@hufs.ac.kr, swlee69@snu.ac.kr, yieom@skku.edu

A Study in Transaction Blocking Time by Lock Contention

Hye-jin Yoo[○] Sung-jae Moon[○] Jong-hyeok Park^{*} Sang-won Lee⁺ Young-ik Eom[○]

Sungkyunkwan University[○], Hankuk University of Foreign Studies^{*}, Seoul National University⁺

요 약

DBMS은 여러 트랜잭션을 병렬적으로 처리하여 성능을 향상시키는 동시에 데이터의 무결성과 트랜잭션 간의 독립성을 보장해야 하는데, 이를 위해 락 기반의 동시성 제어를 사용한다. 락 경합이 발생할 경우, 트랜잭션 대기 시간이 발생하며 이는 트랜잭션의 처리 속도 하락으로 이어진다. 본 논문에서는 MySQL/InnoDB와 TPC-C 벤치마크로 다양한 환경에서 락 경합에 의한 대기 시간을 측정하여 그 문제점을 규명한다. DRAM 버퍼를 최대로 활용한 경우에도 상당한 대기 시간이 발생하였으며 Disk I/O의 영향을 받게 된 경우에는 그에 따라 대기 시간이 상승하였다. 또한, 락 경합을 최소화한 상황에서 대기 시간 감소를 측정하여 연구의 필요성에 대해 제안하였다.

1. 서 론

DBMS(Database Management System)는 여러 트랜잭션을 병렬적으로 수행함으로써 트랜잭션의 처리 성능을 향상시킨다. DBMS는 반드시 트랜잭션 간의 유효성과 독립성을 보장하는 환경 아래에서 트랜잭션의 처리 순서를 정해야만 한다. DBMS의 스케줄러는 트랜잭션 간의 데이터 오염을 막기 위하여 락 기반의 동시성 제어 기법을 사용한다.

락에는 크게 공유 락(Shared Lock)과 배타 락(Exclusive Lock)이 있다. 트랜잭션이 데이터를 읽을 때는 공유 락을 획득하고, 업데이트나 삭제 같은 쓰기 작업을 수행할 때는 배타 락을 획득한다. 데이터를 어떤 트랜잭션이 읽기 또는 쓰기를 하고 있는 데이터에 대해 다른 트랜잭션이 락을 요청하였을 때, DBMS는 락의 호환성(Compatibility)를 고려하여 락을 허용할지를 결정한다. 공유 락이 걸려있는 데이터에 대해서는 공유 락을 획득할 수 있지만, 그 외의 경우는 불가능하여 락이 충돌하게 된다. 만약 배타 락이 설정된 데이터에 대해 다른 트랜잭션이 배타 락을 요청하게 되는 경우처럼, 락이 충돌하는 상황을 락 경합(Lock Contention) 상황이라 일컫는다. 이때 후행 트랜잭션이 요청한 락은 선행 트랜잭션이 보유한 락에 의해 충돌한다. 충돌한 후행 트랜잭션은 선행

락이 해제될 때까지 대기하게 된다. 락 경합으로 인하여 트랜잭션이 대기하게 되면 그에 잇따른 여러 트랜잭션의 수행 시간이 그만큼 길어지게 되며 심각한 성능 저하를 발생시킨다.

본 논문은 락 경합으로 인해 발생하는 대기 문제의 심각성에 대해 규명한다. 또한, 락 경합에 의한 대기 시간이 데이터베이스 성능에 미치는 영향을 분석하기 위해 MySQL/InnoDB를 이용해 실험을 수행한다. 대표적인 OLTP(On-Line Transaction Processing) 벤치마크인 TPC-C를 수행하여 각 트랜잭션의 락 충돌에 의한 대기 시간과 IOPS (Input/Output Per Second) 를 측정한다. 해당 실험 결과를 바탕으로 락 경합이 데이터베이스 성능에 미치는 영향을 분석한다. 또한, 분석 결과를 토대로 대기 시간 문제를 해결할 수 있도록 향후 연구 방향을 제시한다.

2. 배경

2.1 락 경합에 대한 선행 연구 분석

락 경합에 의한 트랜잭션 성능 저하를 개선하기 위해 많은 연구들이 있어왔다. 그중 오랫동안 가장 많이 연구된 기법은 ‘Early Lock Release’ 기법으로, 트랜잭션이 메모리 상 로그 버퍼에 공간을 할당 받은 즉시 락을 해제한다 [1]. 즉, 레코드가 스토리지에 플러시되기 전에 락을 해제하여 그 쓰기 시간만큼 락 경합 대기 시간을 줄일 수 있다. 하지만 Early Lock Release 기법은 잘못된

이 성과는 정부(과학기술정보통신부)의 재원으로 한국연구재단의 지원을 받아 수행된 연구임(No. 2022R1A2C2008225).

쓰기 결과를 낼 수 있다는 치명적인 문제가 있어 태그와 같은 추가적인 보조 요소를 이용하여 문제를 해결했다.

Early Lock Release 기법의 단점들을 보완하여 제안된 ‘Controlled Lock Violation’ 기법에서는 락을 해제하는 대신 유지하되, 위반하는 방식을 제안한다 [2]. 플러시 과정 동안 락을 해제하여 락 경합을 막았던 Early Lock Release와 달리, 락을 계속 유지하면서 충돌 상황을 무시하고 트랜잭션을 진행시킨다. 이 위반 작업은 매우 제어된 환경에서만 가능하며, 락을 유지하기 때문에 잘못된 쓰기 문제에서도 자유롭다. 태그 같은 보조 요소의 필요성도 없을뿐더러 분산 데이터베이스 환경 등 여러 환경의 데이터베이스를 고려하여 최적화가 가능하기 때문에 안정적인 성능 향상을 이끌어 냈다.

2.2 락 경합과 트랜잭션 대기 시간

위와 같은 선행연구로 볼 수 있듯이, 데이터베이스의 동시성 제어 분야에서 성능 문제는 주로 DRAM 버퍼와 스토리지의 접근 속도 차이에서 발생하였다. 하드 디스크보다 더 빠른 쓰기 속도를 보이는 플래시 메모리 기반 저장장치도 버퍼와의 속도 차이는 극명하기 때문이다. 따라서 이러한 차이를 최소화하며 데이터의 영구성을 보장하는 연구가 현재 활발히 진행 중이다 [3].

하지만, 로그 커밋에 따른 오버헤드를 제외하더라도, 커밋 이전 트랜잭션 진행 중의 락 경합에 의한 대기 시간은 여전히 문제로 남아 있다. MySQL은 이러한 대기 시간을 최소화하기 위하여 8.0.20 버전부터 CATS(Contention-Aware Transaction Scheduling) 알고리즘을 트랜잭션 스케줄링에 도입하기도 하였다 [4]. 기존의 FIFO(First In First Out) 알고리즘에서는 락을 요청한 순서대로 획득을 허용하였으나, CATS 알고리즘에서는 대기하고 있는 트랜잭션에 우선순위를 부여하여 처리 속도를 개선하였다. 최악의 경우에는 락이 과도하게 쌓이면서 데드락 (Deadlock) 으로 판단되어 트랜잭션이 롤백할 수도 있다 [5].

3. 실험

3.1 실험 환경

본 논문에서는 락 경합에 따른 대기 시간을 분석하기 위해 표 1과 같은 환경으로 실험을 진행하였다. DBMS는 가장 대중적으로 사용되는 MySQL/InnoDB를 사용하였고, 대표적인 OLTP 워크로드인 TPC-C 벤치마크를 사용하여 표 2와 같은 실험을 수행하였다. 버퍼 메모리에 최대한 많은 페이지를 캐싱해두어 스토리지 접근 시간을 최소화하기 위하여 램프업(Ramp-Up) 시간은 500초로 설정하였

다.

표 1 시스템 환경

OS	Ubuntu 18.04.6 LTS
CPU	Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz
Memory	32GB
Kernel	4.15.0-213-generic
DBMS	MySQL-5.7.24
Data Device	Samsung SSD 970 PRO 1TB
Log Device	Samsung SSD 840 PRO Series

표 2 실험 별 벤치마크 환경

	실험 1	실험 2
warehouse	10	
Benchmark	tpcc-mysql	
Ramp-Up	500 sec	
Runtime	1200 sec	
Buffer Pool Size	2GB, 512MB, 256MB	2GB
Connection	8	32

3.2 실험1: 버퍼 풀 크기 조정 실험 결과

먼저, 다양한 크기의 버퍼 풀에서 락 경합에 따른 대기 시간을 확인하는 실험을 진행하였다. 이를 위해, TPC-C warehouse 10개인 1GB를 기준으로 200%인 2GB, 50%인 512MB, 25%인 256MB 크기의 버퍼 풀에서 트랜잭션 간의 대기 시간을 측정하여 변화를 살펴보았다.

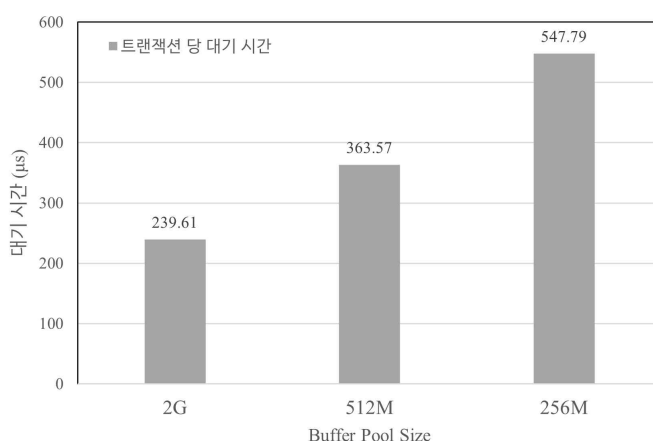


그림 1 버퍼 풀의 크기에 따른 대기 시간

표 3 Buffer Pool Size에 따른 데이터베이스 성능

	트랜잭션 당 대기 시간	IOPS
2G	239.61 μ s	2,398
512M	363.57 μ s	4,815
256M	547.79 μ s	7,000

2GB의 버퍼 풀의 경우에는 데이터의 200% 크기에 해당하여 스토리지 접근 횟수가 적은 상황임에도 락 경쟁으로 인하여 대기 시간을 발생시키고 있다. 이는 512MB와 비교하였을 때도 적지 않은 시간임을 알 수 있다. 버퍼 풀의 크기가 감소할수록 대기 시간이 증가한 것은 IOPS가 증가한 것을 보아, 스토리지 접근에 따른 선행 락 보유 시간이 길어짐에 따라 후행 락 또한 그만큼 대기하게 되어 최종적으로 대기 시간이 증가하게 된다.

3.3 실험2: Payment 쿼리 순서 조정 실험 결과

다음으로는, 실험 1과 다르게 커넥션 개수를 32개로 설정하여 락 경쟁을 극대화 시켜 대기 시간이 트랜잭션 처리 성능에 더 큰 영향을 미치게 하였다. 그리고 이때 TPC-C 임의로 쿼리 순서를 조정하여 충돌을 줄였을 때, 대기 시간의 개선을 확인하였다. TPC-C의 Payment 트랜잭션은 전체 트랜잭션 처리 시간 중 가장 많은 시간을 차지하므로 충돌로 인한 대기 시간에 가장 많은 영향을 받는다. Payment에서 업데이트하는 Warehouse 테이블의 경우 데이터의 양이 테이블 중 가장 적고 접근이 많으므로 락 경쟁이 발생할 확률이 크다. 그러므로 Warehouse에 대한 쓰기 작업의 순서를 뒤로 변경하면 락 경쟁에 의한 대기 시간을 줄일 수 있다 [6].

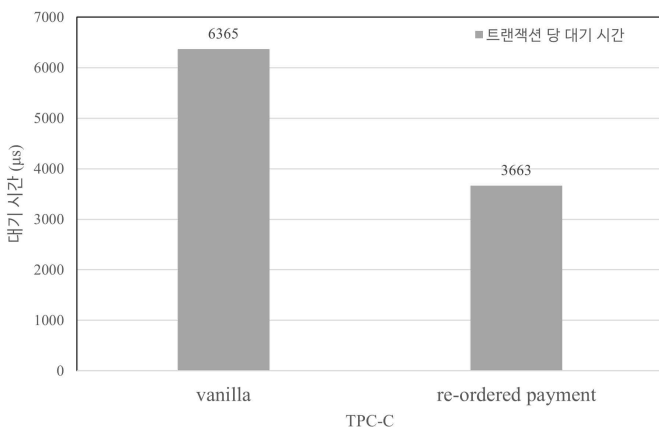


그림 2 쿼리 순서 조정에 따른 대기 시간

Payment 트랜잭션의 쿼리 순서를 변경하였을 때, 트랜잭션 당 평균 대기 시간이 크게 하락하였다. 이는 쓰기가 자주 일어나는 테이블에 대한 쓰기 작업의 순서를 뒤로 조정함으로써 락 경쟁이 더 짧게 발생하여 그에 따른 트랜잭션의 대기 시간이 감소한 것이다.

4. 결론 및 향후 연구

본 논문에서는 락 경쟁으로 인한 트랜잭션의 대기 시간이 다른 트랜잭션의 처리 속도를 늦추고, Disk I/O의 영향까지 받게 될 경우 더 오래 대기하면서 트랜잭션의 전체 성능에 큰 영향을 준다. 뿐만 아니라, 락 경쟁을 최소화하였을 때의 성능 차이를 분석하여 성능 개선의 가능성을 제안하였다. 향후에는 Disk I/O가 적은 상황에서도 성능을 하락시키는 락 경쟁 문제를 해결하여 트랜잭션을 보다 동시적으로 수행하는 방법을 모색할 예정이다.

5 참고문헌

- [1] Hideaki Kimura, Goetz Graefe, “Harumi Kuno: Efficient locking for databases on modern hardware”, ADMS workshop, 2012
- [2] Goetz Graefe, Mark Lillibridge, Harumi Kuno, Joseph Tucek, and Alistair Veitch, “Controlled lock violation”, ACM SIGMOD International Conference on Management of Data, 85-96, 2013
- [3] Mijin An, Jonghyeok Park, Tianzheng Wang, Beomseok Nam, and Sang-Won Lee, “NV-SQL: Boosting OLTP Performance with Non-Volatile DIMMs”, Proc. VLDB Endow, 16, 6, 1453-1465, 2023
- [4] MySQL Team, “Changes in MySQL 8.0.20,” <https://dev.mysql.com/doc/relnotes/mysql/8.0/en/news-8-0-20.html>, 2020
- [5] MySQL. Team, “Mysql 8.0 reference manual,” <https://dev.mysql.com/doc/refman/8.0/en/>, 2023.
- [6] Cong Yan and Alvin Cheung, “Leveraging lock contention to improve OLTP application performance”, Proc. VLDB Endow, 9, 5, 444-455, 2016